Ders 22 Genişletilmiş Notu: Uygulama: exec, atexit, Zombi ve Öksüz Süreçler, wait, Pipe, FIFO ve Sinyaller

Ders 22 Genişletilmiş Notu: Uygulama: exec, atexit, Zombi ve Öksüz Süreçler, wait, Pipe, FIFO ve Sinyaller

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Oturumun Başlangıcı ve exec'e Dönüş
- 2. exec Örneği
- 3. Süreç Sonlandırma: exit, atexit ve return
- 4. Zombi ve Öksüz Süreçler
- 5. wait ile Ebeveyn ve Çocuğu Eşzamanlamak
- 6. Süreçler Arası İletişim ve İsimsiz Boru (pipe)
- 7. İsimli Boru (FIFO)
- 8. Sinyaller
- Örnek Sorular
- Dersin Toparlaması

1. Oturumun Başlangıcı ve exec'e Dönüş

Bu kayıt bir uygulama (laboratuvar) oturumudur ve çevrim içi yapıldı. Dr. Ziya Cihan Tayşi oturumu açtı, yetkiyi uygulamayı anlatacak öğretim elemanına verdi ve ayrıldı. Bu notta "öğretim elemanı" dendiğinde uygulamayı anlatan kişi kastedilir.

Bir önceki uygulama oturumunda ciddi bağlantı sorunları yaşanmıştı. Öğretim elemanı bu oturumun biraz onun telafisi olacağını, yine de kaldığı yerden devam edeceğini söyledi. Kaldıkları yer `exec` idi.

Öğretim elemanı önce `fork` ile `exec` arasındaki farkı hatırlattı. `fork` ile oluşturulan çocuk süreç (child process), ebeveynin (parent) birebir kopyasıdır. `exec` ise çocuğun farklı bir iş yapmasını sağlar. Çocuk kendisinin kopyası olarak kalmaz, orada bambaşka bir program çalıştırılır.

2. exec Örneği

Örnekte `fork`'un dönüş değeri bir `pid` değişkenine atanır. `fork` ebeveyne oluşturulan çocuğun PID'sini (process identifier) döndürür, çocuğa ise 0 döndürür. Bu yüzden:

- `pid > 0` ise ebeveyn sürecin içindeyiz.
- `pid == 0` ise çocuk sürecin içindeyiz.

Örnekte ebeveyn yalnızca "something completely different" gibi bir mesaj basıyor. Çocuk ise `exec` ile `ls` programını çalıştırıyor. Aşağıdaki kod, derste anlatılan yapıyı kuran ve `gcc -Wall` ile derlenip çalıştırılmış bir sürümdür. Ekrandaki kodun tamamı altyazıdan okunamıyor.

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid > 0) {                          /* ebeveyn */
        printf("Parent: something completely different\n");
    } else {                                /* çocuk */
        execlp("ls", "ls", (char *)NULL);
        perror("execlp");                   /* yalnız exec başarısızsa çalışır */
        exit(1);
    }
    wait(NULL);
    return 0;
}
```

Derste program çalıştırıldığında önce `ls` çıktısının, ardından ebeveynin mesajının göründüğü anlatıldı. Öğretim elemanı çıktı sırasını anlatırken birkaç kez düzeltme yaptı.

> **Ders sonrası not:** Öğretim elemanı çocuğun `exec` ile "kendi çocuğunu oluşturduğunu" söyledi. Doğrusu: `exec` yeni süreç oluşturmaz. Çağıran sürecin bellek görüntüsünü yeni programla değiştirir, PID aynı kalır. `exec` başarılı olursa ondan sonraki satırlar hiç çalışmaz. Ayrıca ebeveyn ile çocuğun çıktılarının sırası zamanlayıcıya bağlıdır, sabit değildir. Bizim denememizde ebeveynin satırı `ls` çıktısının sonunda göründü, ama bu sıra garanti değildir.

3. Süreç Sonlandırma: exit, atexit ve return

3.1 exit(0)

Süreç sonlandırmada `exit` fonksiyonuna verilen değer, sürecin çıkış durumudur (exit status). `exit(0)` yazmak, sürecin bütün işlemlerini normal şekilde yapıp bittiği anlamına gelir.

3.2 atexit Örneği

`atexit`, süreç normal sonlanırken çağrılacak bir fonksiyonu kaydeder. Örnekte `parent_cleaner` adında bir fonksiyon yazılmış ve `atexit`'e parametre olarak verilmiş. Süreç çıkarken bu fonksiyon ne yapıyorsa o yapılır.

Örnekte `atexit` kaydı ebeveynin içinde yapılıyor. Çocuk ise kendi işini yapıp `exit(0)` ile çıkıyor. Öğretim elemanı önce "çocuk biterken de `atexit`'teki fonksiyon çağrılacak mı?" diye sordu, sonra çıktıya baktı. Çıktıda ebeveynin PID'si, ebeveyn biterken çağrılan temizleyici fonksiyonun mesajı ve çocuğun satırı görüldü. Çocuk `exit(0)` ile normal şekilde sonlandı.

Derste anlatılan yapıyı kuran kod:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void parent_cleaner(void) {
    printf("parent_cleaner: PID %d çıkıyor\n", getpid());
}

int main(void) {
    pid_t pid = fork();
    if (pid > 0) {                          /* ebeveyn */
        atexit(parent_cleaner);             /* yalnız ebeveynde kaydedildi */
        printf("Parent PID %d\n", getpid());
    } else if (pid == 0) {                  /* çocuk */
        printf("Child PID %d\n", getpid());
```

```
        exit(0);
    }
    return 0;
}
```

Bu kod derlenip çalıştırıldığında temizleyici fonksiyon yalnız ebeveynin PID'siyle bir kez çalıştı. Çünkü kayıt `fork`'tan sonra ve yalnız ebeveyn dalında yapıldı.

> **Ders sonrası not:** `atexit` kaydı `fork`'tan önce yapılırsa çocuk bu kaydı devralır. Çocuk `exit` ile çıktığında aynı fonksiyon çocukta da çalışır. Denememizde fonksiyon `fork`'tan önce kaydedilince iki farklı PID ile iki kez çalıştı. Çocukta bunu istemiyorsanız çocuk `_exit` ile çıkmalıdır.

3.3 exit ile return Farkı

Öğretim elemanı `return` ile `exit` arasındaki farkı sordu ve "`exit` bir sistem çağrısı (system call), `return` ise bir komut (instruction)" diye cevapladı.

> **Hocanın vurgusu:** `exit` ile `return` farkı doğru-yanlış sorularında karşınıza çıkabilir. Aklınızda tutun.
>
> **Ders sonrası not:** Öğretim elemanının "`exit` sistem çağrısıdır" ifadesi tam doğru değil. `exit()` bir C kütüphane fonksiyonudur. `atexit` ile kaydedilen fonksiyonları çalıştırır, `stdio` tamponlarını boşaltır, sonra asıl sistem çağrısı olan `_exit`'i yapar. `return` ise yalnız bulunduğu fonksiyondan döner. `main`'den `return` etmek `exit`'i çağırmakla aynı etkiyi yapar. Başka bir fonksiyondaki `return` süreci sonlandırmaz.

4. Zombi ve Öksüz Süreçler

4.1 Tanımlar

Öğretim elemanı bir önceki oturumda bağlantının tam bu konuda koptuğunu söyledi ve zombileri baştan anlattı. Ona göre zombiler iki şekilde oluşur, birine öksüz (orphan) da denir.

- **Zombi süreç (zombie process):** Çocuk süreç bitmiştir. Ama ebeveyn bu bitişin onayını vermemiştir. Bu yüzden sistem, çocuğun kaydını süreç tablosundan (process table) silemez. Çocuk aslında sonlanmış (terminated) durumdadır, ama ebeveynden onay alamadığı için tabloda durmaya devam eder.
- **Öksüz süreç (orphan process):** Çocuk bir şey beklemektedir ya da çalışmaya devam etmektedir, ama ebeveyn sonlanmıştır. Ebeveyn bitmiş, çocuk ortada kalmıştır. Öğretim elemanı bunun da bir zombi süreci ortaya çıkardığını söyledi.

  > **Ders sonrası not:** Öksüz süreç zombinin bir türü değildir, ikisi farklıdır. Zombi: çocuk sonlanmış, ebeveyn `wait` yapmamıştır. Öksüz: ebeveyn sonlanmış, çocuk hâlâ çalışıyordur. Öksüz süreci `init` (günümüz Linux'unda `systemd` ya da bir alt toplayıcı süreç) sahiplenir. Çocuk sonlanınca durumunu bu yeni ebeveyn toplar, bu yüzden öksüz süreç zombi olarak kalmaz.

4.2 Birinci Zombi Örneği

Birinci örnekte çocuk hemen sonlanır. Ebeveyn ise `wait` yapmadan sonsuz bir döngüye girer, bu yüzden hiç bitmez. Çocuğun bitiş durumu kimse tarafından alınmadığı için çocuk zombi olur.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    if (fork() == 0) {                    /* çocuk hemen biter */
        printf("Terminating Child, PID = %d\n", getpid());
        exit(0);
    } else {                              /* ebeveyn wait yapmadan döner */
```

```
        printf("Running Parent, PID = %d\n", getpid());
        while (1)
            ;
    }
}
```

Öğretim elemanı program çalışırken süreçleri `ps` ile listeledi. Çıktı uzun olduğu için `grep` ile süzmenin en mantıklısı olduğunu söyledi ve durumu (status) büyük `Z` olan süreci aradı. Çıktıda sütunlar sırasıyla PID ve durum idi. Çocuk sürecin durumunun `Z` olduğu, yani zombi olduğu görüldü.

Zombinin nasıl öldürüleceğine de baktığını anlattı. Ebeveyne sinyal gönderilebilir, ama ebeveyn doğru kodlanmamışsa bu pek işe yaramaz. Bu durumda `kill` ile ikisini birden öldürmek zorunda kalınabilir.

> **Ders sonrası not:** Zombi süreç zaten sonlanmıştır, ona `kill` göndermek bir şey değiştirmez. Zombiyi kaldırmak için ebeveynin `wait` yapması gerekir. Ebeveyn öldürülürse zombi `init`'e (ya da alt toplayıcıya) geçer ve hemen toplanır. Denememizde `ps -o pid,ppid,stat` çıktısında çocuk `Z` (`<defunct>`) olarak göründü. Yalnız ebeveyne `kill` gönderildiğinde zombi de tablodan kalktı.

4.3 İkinci Örnek (Zombi 2): Öksüz Süreç

İkinci örnekte durum tersidir. Ebeveyn hemen çıkar. Çocuk ise sonsuz bir döngüde bekler, aslında sonsuza kadar bekleyecektir.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    if (fork() == 0) {                    /* çocuk sonsuza dek çalışır */
        printf("Running Child, PID = %d\n", getpid());
        while (1)
            ;
    } else {                              /* ebeveyn hemen çıkar */
        printf("Terminating Parent, PID = %d\n", getpid());
        exit(0);
    }
}
```

Program çalıştırıldığında öğretim elemanı beklediği zombiyi `ps` çıktısında göremedi. Programın "kendi kendine sonlandığını", bu yüzden bir zombi oluşturamadıklarını söyledi. Çocuğun `while` döngüsünden nasıl çıkabildiğini anlayamadığını belirtti ve öğrencilere fikir sordu. Bu tartışma aşağıda "Örnek Sorular" bölümündedir.

> **Ders sonrası not:** Bu sonuç beklenen sonuçtur. Komut istemi geri geldiği için program bitmiş gibi görünür, çünkü kabuk (shell) yalnız ebeveyni bekler. Çocuk döngüden çıkmamıştır. Arka planda öksüz olarak çalışmaya devam eder ve yeni ebeveyni `init` ya da `systemd` olur. Denememizde çocuk `R` (çalışıyor) durumunda göründü. `PPID` sütununda artık eski ebeveyn değil `systemd --user` vardı. Çocuk sonsuz döngüde olduğu için elle öldürülmesi gerekir.

5. wait ile Ebeveyn ve Çocuğu Eşzamanlamak

Öğretim elemanı zombi ve öksüz durumlarının önüne geçmek için `wait` sistem çağrısının kullanıldığını söyledi. Çocuklar sonlandığında ebeveyne bunu haber verir. Ebeveyn `wait` ile bu haberi alınca çocuklar süreç tablosundan silinir. Amaç, çocuk ile ebeveyni eşzamanlı (synchronous) çalıştırmaktır: ebeveyn, bütün çocukları bitmeden kendisi sonlanmasın.

Örnekte çocuk sayısı (`N`) biliniyor. `N` kez `fork` çağrılıyor ve her çocuk farklı bir çıkış durumuyla çıkıyor: i. çocuk $100+i$ ile döner. Yani ilk çocuk 100, ikinci 101, üçüncü 102 ile çıkar. Ebeveyn de `N` kez `wait` çağırır.

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
#define N 3

int main(void) {
    int child_status;
    pid_t pid[N], wpid;
    for (int i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0)
            exit(100 + i);                      /* çocuk i, 100+i ile çıkar */
    for (int i = 0; i < N; i++) {
        wpid = wait(&child_status);             /* biten çocuğun PID'si */
        if (WIFEXITED(child_status))            /* normal mi çıktı? */
            printf("Child %d terminated with exit status %d\n",
                   wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminated abnormally\n", wpid);
    }
    return 0;
}
```

Öğretim elemanı kodun parçalarını şöyle açıkladı:

- `wpid`: `wait`'in döndürdüğü değerdir, yani biten çocuğun PID'si.
- `child_status`: `wait`'in doldurduğu durum bilgisidir. Bir çocuk `exit` yaptıysa ebeveyne bu durum döner. Bu bilgi elde edildiyse çocuk sonlanmış demektir.
- `WIFEXITED(child_status)`: Çocuğun normal şekilde çıkıp çıkmadığını söyleyen makrodur. Normal çıktıysa sıfırdan farklı (doğru) döner.
- `WEXITSTATUS(child_status)`: Çocuğun çıkış durumunu verir. Bu makroyla çıkış kodu ekrana bastırılabilir.

Derlenip çalıştırılan sürümde üç satır basıldı. Her satırda biten çocuğun PID'si ile 100, 101 ve 102 çıkış durumları vardı. Öğretim elemanı da anlatımın sonunda "çok karışık anlattım" dedi. Özetle akış şudur: her `wait` bir çocuğun bitmesini bekler, PID'sini döndürür ve durumunu `child_status`'a yazar; makrolar bu durumu çözer.

> **Ders sonrası not:** Öğretim elemanı iki noktada hatalı ifade kullandı. Birincisi, `wait(&child_status)` yerine `wait(NULL)` yazılırsa "bütün çocukların bekleneceğini" söyledi. Doğrusu: `wait(NULL)` herhangi bir çocuğun bitmesini bekler ve durumu yok sayar. Çocuk sayısı bilinmiyorsa bütün çocukları beklemek için `while (wait(NULL) > 0);` gibi bir döngü kurulur; çocuk kalmayınca `wait` -1 döndürür. İkincisi, "çıkış durumu ebeveyne ulaşana kadar ebeveyn çıkamaz, kural bu" dedi. Böyle bir kural yoktur. Ebeveyn istediği an sonlanabilir, o zaman çocuklar öksüz kalır. `wait` yalnızca ebeveynin kendi isteğiyle beklemesini sağlar.

6. Süreçler Arası İletişim ve İsimsiz Boru (pipe)

6.1 Genel Bakış

Öğretim elemanı uygulamanın asıl amacının süreçler arası iletişim (inter-process communication, IPC) olduğunu, önceki oturumda buraya hiç giremediklerini söyledi. Slayttaki IPC yöntemlerinden dördünü göreceklerini, semaforların (semaphore) ise bir sonraki hafta başka bir öğretim elemanı tarafından anlatılacağını belirtti.

Boru (pipe) ve FIFO ile süreçler arasında tek yönlü ama eşzamanlı (synchronous) bir iletişim kurulur. İkisinin farkı:

- **Boru (pipe):** İsimsiz boru (unnamed pipe) da denir. Ebeveyn ile çocuk arasında kullanılır.
- **FIFO:** İsimli boru (named pipe) da denir. Böyle bir akrabalık şartı yoktur.

Ders sonrası not: Daha doğru ifade: isimsiz boru, ortak bir atası olan ve boruyu

`fork` ile miras alan süreçler arasında kullanılır. Bu ebeveyn-çocuk olabileceği gibi iki kardeş süreç de olabilir.

6.2 Boru Kavramı

Öğretim elemanı slayttaki şekli göstererek boruyu gerçek bir boru gibi düşünmemizi istedi. Borunun bir okuma ucu (read end) ve bir yazma ucu (write end) vardır. Süreçlerden biri yalnız okuma ucunu, diğeri yalnız yazma ucunu kullanır. Veri yalnızca bir yönde akar. Bu özelliğe yarı çift yönlü (half duplex) denir.

Boru `pipe` sistem çağrısıyla oluşturulur. `pipe`, iki elemanlı bir tamsayı dizisi alır. Bu iki eleman dosya tanımlayıcılarıdır (file descriptor), yani borunun iki ucudur:

- `fd[0]`: okuma ucu.
- `fd[1]`: yazma ucu.

`pipe` -1 döndürürse bir sorun oluşmuş demektir.

6.3 Birinci Örnek: Çocuktan Ebeveyne Sayı Gönderme

Bu örnekte çocuk yazar, ebeveyn okur. Çocuk okuma ucunu kullanmayacağı için onu kapatır. Kullanıcıdan bir sayı ister, `write` ile bu sayıyı yazma ucuna bir `int` boyu kadar yazar ve yazma ucunu da kapatır. Ebeveyn bunun tersini yapar. Yazma ucunu kapatır, `read` ile okuma ucundan bir `int` boyundaki veriyi `y` değişkenine okur, okuma ucunu kapatır ve sonucu ekrana basar.

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    int fd[2];                          /* fd[0]: okuma ucu, fd[1]: yazma ucu */
    if (pipe(fd) == -1) { perror("pipe"); exit(1); }
    pid_t pid = fork();
    if (pid == 0) {                     /* çocuk: yazar */
        close(fd[0]);
        int x;
        printf("Enter a number: ");
        fflush(stdout);
        scanf("%d", &x);
        write(fd[1], &x, sizeof(int));
        close(fd[1]);
    } else {                            /* ebeveyn: okur */
        close(fd[1]);
        int y;
        read(fd[0], &y, sizeof(int));
        close(fd[0]);
        printf("Got from child process %d\n", y);
        wait(NULL);
    }
    return 0;
}
```

Çalıştırınca önce çocuktaki “Enter a number” mesajı göründü. Çocuk girilen sayıyı borunun yazma ucuna yazdı, ebeveyn de okuma ucundan okuyup bastı. Denememizde 42 girildi ve ebeveyn 42’yi bastı.

6.4 İkinci Örnek: Boru ile İstemci-Sunucu

Öğretim elemanı bunu “daha afilli” bir örnek olarak tanıttı. Klasörde `ourpipe` adlı bir metin dosyası vardı. Programın amacı şudur: istemci (client) bir dosya adı söyler, sunucu (server) da

o dosyanın içinde ne varsa ekrana basar. Yani iki süreç birbirine dosyanın adını iletir, içeriği ise sunucu ekrana yazar.

Örnekte çocuk süreç sunucudur ve borunun okuma ucunu kullanır. Ebeveyn istemcidir ve yazma ucunu kullanır.

Sunucu tarafında `read`'in dönüş değeri önemlidir:

- `read` 0 döndürürse dosya sonu (end of file) demektir. Bu durumda sunucu "end-of-file while reading pathname" mesajı basar.
- `read` negatif (-1) döndürürse bir sorun olmuştur, okunamamıştır.
- `read` n döndürürse n bayt okunmuştur.

Her şey yolunda giderse n, tampon boyutu olan `MAXLINE + 1`'den küçük bir sayıdır. Tamponun hepsi kullanılmamış olabileceği için okunan verinin sonuna null karakter (`'\0'`) yazılır. Böylece dizginin nerede bittiği anlaşılır.

Sonra sunucu, tampondaki dosya adını `open` ile açar. Yani istemcinin söylediği dosyayı açar. `open`'dan sıfırdan küçük bir değer dönerse sorun var demektir ve hata basılır. Sorun yoksa sunucu, $n > 0$ olduğu sürece, yani okunacak bir şey kaldıkça, dosyadan okur ve okuduğu n baytı `STDOUT_FILENO`'ya yazar. `STDOUT_FILENO` da bir dosya tanımlayıcısıdır. Ekranın dosya tanımlayıcısıdır ve sistemde varsayılan olarak tanımlıdır. En sonunda dosya kapatılır.

İstemci bir tampon tanımlar. Klavyeden girilen dosya adını tampona alır, uzunluğunu bulur ve `write` ile borunun yazma ucuna bu uzunluk kadar veri yazar.

```c
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>
#define MAXLINE 4096

void server(int readfd) {
    char buff[MAXLINE + 1];
    ssize_t n = read(readfd, buff, MAXLINE);     /* dosya adını borudan oku */
    if (n == 0) {
        fprintf(stdout, "end-of-file while reading pathname\n");
        exit(1);
    }
    buff[n] = '\0';                              /* dizgi sonu: null karakter */
    int fd = open(buff, O_RDONLY);
    if (fd < 0) {
        printf("error: can't open %s\n", buff);
        exit(1);
    }
    while ((n = read(fd, buff, MAXLINE)) > 0)    /* okunacak bir şey kaldıkça */
        write(STDOUT_FILENO, buff, n);           /* ekrana yaz */
    close(fd);
}

void client(int writefd) {
    char buff[MAXLINE];
    printf("Enter filename: ");
    fflush(stdout);
    if (fgets(buff, MAXLINE, stdin) == NULL)     /* klavyeden dosya adı */
        return;
    size_t len = strlen(buff);
    if (len > 0 && buff[len - 1] == '\n')
        len--;                                   /* satır sonunu at */
    write(writefd, buff, len);                   /* borunun yazma ucuna yaz */
}
```

```c
int main(void) {
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); exit(1); }
    pid_t pid = fork();
    if (pid == 0) {                              /* çocuk: sunucu, okur */
        close(fd[1]);
        server(fd[0]);
        close(fd[0]);
        exit(0);
    }
    close(fd[0]);                                /* ebeveyn: istemci, yazar */
    client(fd[1]);
    close(fd[1]);
    wait(NULL);
    return 0;
}
```

Programı çalıştırınca önce dosya adı soruldu, `ourpipe` dosyasının adı girildi ve dosyanın içeriği ekrana basıldı. Öğretim elemanı mesajları hangi sürecin bastığını önce karıştırdı, sonra düzeltti: dosya adını soran istemcidir (ebeveyn), dosyanın içeriğini ekrana basan sunucudur (çocuk). Denememizde var olan bir dosyanın içeriği basıldı. Olmayan bir dosya adında sunucu açma hatası verdi. Boş girişte de sunucu dosya sonu mesajını bastı.

7. İsimli Boru (FIFO)

7.1 mkfifo ve Mod

FIFO'ya isimli boru (named pipe) da denir ve `mkfifo` ile oluşturulur. `mkfifo` bir yol ve bir mod alır. Öğretim elemanının senaryosunda bir sunucu süreç bir yerde, bir istemci süreç başka bir yerde çalışır. İkisi de yine bir boru kullanır. Ancak FIFO'yu "yabancı" bir süreç de kullandığı için erişim modu ayarlanabilir. Bu mod, `chmod`'daki okuma, yazma ve çalıştırma (read, write, execute) izinlerinin aynısıdır ve makrolarla tanımlanmıştır. FIFO açılırken de yalnız yazma (`O_WRONLY`) ya da yalnız okuma (`O_RDONLY`) gibi bayraklar kullanılır.

Örnekteki FIFO'lar `/tmp` altında, `/tmp/fifo.1` ve `/tmp/fifo.2` adlı dosyalar olarak oluşur. İş bitip `unlink` çağrıldığında silinirler. Öğretim elemanı örnekteki FIFO modunun kullanıcı, grup ve diğerleri için okuma, yazma ve çalıştırma izni verdiğini söyledi.

> **Ek bilgi (derste anlatılmadı):** `mkfifo` dosya sisteminde `p` türünde özel bir dosya oluşturur; `ls -l` çıktısında satır `p` ile başlar. Veriler diskte saklanmaz, çekirdekteki bir tampondan geçer. Verilen mod, sürecin `umask` değeriyle maskelenir. Örneğin `0666` istenip `umask 022` ise oluşan izin `0644` olur.
>
> **Ders sonrası not:** FIFO'nun `/tmp` altında oluşması FIFO'ya özgü bir kural değildir. FIFO dosyası `mkfifo`'ya verilen yolda oluşur; örnek kodda `/tmp` yolu seçildiği için orada oluşmuştur.

7.2 Sunucu ve İstemci

Sunucu iki FIFO oluşturur. `fifo.1`'i okuma ucu, `fifo.2`'yi yazma ucu olarak açar. İstemci de aynı yollardaki FIFO'lara bağlanır. İkisi arasındaki iletişim bu iki FIFO üzerinden kurulur. Öğretim elemanı bunu şöyle özetledi: biri FIFO'ya yazar, öbürü oradan okur. Her FIFO yine tek yönlüdür. Ama bir yöndeki iş bitince diğer FIFO'da bu kez öbür süreç yazıp ilki okuyabilir.

Sunucu `fifo.1`'den dosya adını okur. Dosya sonu gelirse yine "end-of-file" mesajı basar. Sorun yoksa dosyayı açar, içeriğini okur ve bu kez yazma FIFO'suna yazar. Öğretim elemanı farkı vurguladı: boru örneğinde sunucu içeriği standart çıkışa yazıyordu, burada FIFO'nun yazma ucuna yazıyor. İstemci ise dosya adını yazma FIFO'suna uzunluğu kadar yazar. Okuma FIFO'sundan okuduklarını da standart çıkışa, yani ekrana basar.

```c
/* fifo_common.h */
#include <fcntl.h>
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <unistd.h>
#define MAXLINE 4096
#define FIFO1 "/tmp/fifo.1"
#define FIFO2 "/tmp/fifo.2"
#define FILE_MODE (S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH)

/* fifo_server.c */
#include "fifo_common.h"

int main(void) {
    if (mkfifo(FIFO1, FILE_MODE) < 0) { perror("mkfifo " FIFO1); exit(1); }
    if (mkfifo(FIFO2, FILE_MODE) < 0) { perror("mkfifo " FIFO2); unlink(FIFO1); exit(1); }
    int readfd  = open(FIFO1, O_RDONLY);   /* istemci yazma için açana kadar bekler */
    int writefd = open(FIFO2, O_WRONLY);
    char buff[MAXLINE + 1];
    ssize_t n = read(readfd, buff, MAXLINE);
    if (n <= 0) { fprintf(stderr, "end-of-file while reading pathname\n"); exit(1); }
    buff[n] = '\0';
    int fd = open(buff, O_RDONLY);
    if (fd < 0) {
        snprintf(buff + n, sizeof(buff) - n, ": can't open\n");
        write(writefd, buff, strlen(buff));
    } else {
        while ((n = read(fd, buff, MAXLINE)) > 0)
            write(writefd, buff, n);       /* bu kez ekrana değil FIFO'ya */
        close(fd);
    }
    close(readfd);
    close(writefd);
    return 0;
}

/* fifo_client.c */
#include "fifo_common.h"

int main(void) {
    int writefd = open(FIFO1, O_WRONLY);   /* FIFO yoksa hata */
    if (writefd < 0) { perror("open " FIFO1); exit(1); }
    int readfd  = open(FIFO2, O_RDONLY);
    char buff[MAXLINE];
    printf("Enter filename: ");
    fflush(stdout);
    if (fgets(buff, MAXLINE, stdin) == NULL) buff[0] = 0;
    size_t len = strlen(buff);
    if (len > 0 && buff[len - 1] == '\n') len--;
    write(writefd, buff, len);
    ssize_t n;
    while ((n = read(readfd, buff, MAXLINE)) > 0)
        write(STDOUT_FILENO, buff, n);
    close(readfd);
    close(writefd);
    unlink(FIFO1);                         /* FIFO dosyalarını sil */
    unlink(FIFO2);
    return 0;
}
```

7.3 Çalıştırma ve Karşılaşılan Sorun

Öğretim elemanı sunucuyu ve istemciyi iki ayrı terminalde çalıştırdı. İstemci, FIFO'nun öbür ucu bağlanana kadar "sabırla" bekledi. Dosya adı olarak `ourpipe.txt` girildi. Ekranda dosya adının sunucuya gönderildiğini söyleyen mesaj çıktı ve istemci, öbür taraf bağlanana kadar beklemeye devam etti. Öğretim elemanı bu bekleyişin eşzamanlı iletişimin sonucu olduğunu söyledi.

Ancak iletişim bir türlü kurulmadı. Bir öğrenci sorunun nedenini fark etti: istemci önce çalıştırılmıştı. FIFO'ları sunucu oluşturuyorsa ve sunucu henüz çalışmamışsa, istemcinin bağlanacağı FIFO yoktur. Öğretim elemanı istemcide `mkfifo` olmadığını kontrol etti ve öğrenciyi doğruladı: FIFO'yu oluşturan sunucudur, istemci yalnız kullanır. "Oluşmamış şeyi nasıl okuyacak?" diyerek bunun güzel bir nokta olduğunu söyledi.

Son olarak `/tmp` klasörünü gösterdi. İletişim sırasında orada FIFO dosyaları oluşmuştu, iş bitince silinmişlerdi. FIFO, geçici bir klasörde bir dosya oluşturur ve ona ne yazılırsa öbür taraf onu okur. Bunlar için bazı standartlar oturtulduğu için süreçler arasındaki iletişim çok daha kolay kurulur.

Denememizde önce istemci çalıştırılınca `open` "No such file or directory" hatası verdi. Sunucu önce çalıştırılınca `/tmp/fifo.1 p` türünde bir dosya olarak oluştu. Dosya içeriği istemcinin ekranına basıldı ve iş bitince iki FIFO dosyası da silindi.

8. Sinyaller

8.1 Eşzamansız İletişim ve Üç Tepki

Boru ve FIFO'da iki tarafın da iletişime bağlı olması bekleniyordu. Öğretim elemanı bunların eşzamanlı olduğunu, eşzamansız (asynchronous) iletişimin örneğinin ise sinyaller (signals) olduğunu söyledi. "Meşhur `kill` sinyali" bunlardan biridir.

Bir sürece sinyal gönderildiğinde süreç bu sinyali üç şekilde ele alabilir (handle):

- **Yok sayma (ignore):** Sinyal umursanmaz.
- **Sinyal işleyici fonksiyon (signal handler function):** Kullanıcının yazdığı bir fonksiyon çalıştırılır.
- **Varsayılan eylem (default action):** Sinyal gelince varsayılan olarak ne yapılıyorsa o yapılır.

Örneğin Ctrl+C'ye basınca çalışan program durur. Sinyal işlemeyle, yani `sigaction` ile oynamazsanız bu varsayılan davranış geçerlidir.

> **Ders sonrası not:** Öğretim elemanı Ctrl+C'nin bir "termination sinyali" gönderdiğini söyledi. Ctrl+C `SIGINT` (kesme, interrupt) sinyalini gönderir. Sonlandırma sinyali `SIGTERM`'dür. `SIGINT`'in varsayılan eylemi de süreci sonlandırmaktır, karışıklık buradan gelir. Ctrl+Z ise `SIGTSTP` gönderir ve süreci durdurur.

8.2 POSIX ve Sinyal Maskesi

Öğretim elemanı sinyalleri bir standarda oturtmak için POSIX'in kullanıldığını, `sigaction` fonksiyonuyla da kullanıcıya bir miktar özgürlük alanı tanındığını anlattı. Bu özgürlüğün neden gerektiğini bir örnekle açıkladı. Programınızda çok önemli bir kod parçası olsun ve o çalışırken asla kesme (interrupt) gelmesini istemeyesiniz. Örneğin bir hava aracı havalanırken onun elektrik sisteminde koşan yazılıma kesme gelmesini asla istemezsiniz.

> **Hocanın vurgusu:** Sinyalleri engellemenin amacı, kritik bir kod parçasının kesilmeden çalışmasını sağlamaktır.

Bunun için bir sinyal maskesi kullanılır. Sürecin engellenen (blocked) sinyallerinin bir kümesi vardır. Bu kümeye yeni bir sinyal eklenebilir ya da bazı sinyaller bir süreliğine çıkarılıp sonra geri eklenebilir. Örneğin `SIGINT` normalde engellenmemiş sinyaller arasındadır. Biz onu engellemek istiyorsak engellenen kümeye koyarız. Engeli kaldırana kadar bu sinyal bizim belirlediğimiz şekilde ele alınır.

Adımlar şöyledir:

- `sigemptyset(&newmask)` ile boş bir sinyal kümesi oluşturulur. Kümenin adı `newmask`'tır ve adresi gönderilir.
- `sigaddset(&newmask, SIGINT)` ile kesme sinyali bu kümeye eklenir.
- `sigprocmask(SIG_BLOCK, &newmask, &oldmask)` ile `newmask` engellenenlere eklenir. Eski maske `oldmask`'ta saklanır, çünkü sonra lazım olacaktır.
- Kritik kod bölgesi (critical region) çalıştırılır.
- `sigprocmask(SIG_SETMASK, &oldmask, NULL)` ile engellenen sinyaller listesi eski hâline döndürülür.

```
sigset_t newmask, oldmask;

sigemptyset(&newmask);
sigaddset(&newmask, SIGINT);
sigprocmask(SIG_BLOCK, &newmask, &oldmask);   /* SIGINT'i engelle, eskisini sakla */

/* kritik kod bölgesi */

sigprocmask(SIG_SETMASK, &oldmask, NULL);     /* eski maskeye dön */
```

Öğretim elemanı eski maskeyi saklamanın önemini "fabrika ayarları" benzetmesiyle anlattı. Normalde sinyallerin varsayılan sürümlerini kullanmak isteriz, çünkü sistem en uygununu ayarlamıştır. Kendi ihtiyacımız için sinyalleri engelleyip engeli kaldırırken eski ayarı saklarsak, işimiz bitince her şeyi eski hâline, yani fabrika ayarlarına döndürebiliriz.

> **Hocanın vurgusu:** Sinyal ayarlarını değiştirmeden önce eski maskeyi (ya da eski `sigaction` yapısını) saklayın. İşiniz bitince eski hâle dönebilirsiniz.

> **Ders sonrası not:** Öğretim elemanı engellenen sinyalin "bizim dediğimizi yaptığını ya da yok sayıldığını" söyledi. Engellenen sinyal yok sayılmaz. Beklemede (pending) kalır ve engel kalktığı anda teslim edilir. Denememizde `SIGINT` engelliyken `raise(SIGINT)` çağrıldı. `sigpending` sinyalin beklemede olduğunu gösterdi. `SIG_SETMASK` ile eski maskeye dönülünce sinyal teslim edildi ve süreç `SIGINT`'in varsayılan eylemiyle sonlandı. Sinyali gerçekten yok saymak için `SIG_IGN` kullanılır.

8.3 sigaction ile Kendi İşleyicimiz

Sinyali yalnız engelleyip engelini kaldırmak dışında, sinyal geldiğinde yapılacak işi de değiştirebiliriz. Örnekte Ctrl+C geldiğinde, yani kesme sinyali verildiğinde başka bir iş yapılacak.

Bunun için `sigaction` adlı bir yapı (struct) vardır. Öğretim elemanı slayttaki yapının alanlarını saydı: çalıştırılacak fonksiyon, maske olarak bir sinyal kümesi, bayraklar (flags) ve sinyalle ilgili bilgi (info) alan bir fonksiyon.

> **Ek bilgi (derste anlatılmadı):** POSIX'te `struct sigaction`'ın başlıca alanları şunlardır: `sa_handler` (sinyal numarasını alan işleyici, ya da `SIG_IGN`/`SIG_DFL`), `sa_sigaction` (`SA_SIGINFO` bayrağıyla kullanılan, sinyal numarası yanında `siginfo_t *` ve `void *` bağlam işaretçisi alan işleyici), `sa_mask` (işleyici çalışırken ek olarak engellenecek sinyaller) ve `sa_flags`. Derste geçen "info" alanı büyük olasılıkla `sa_sigaction`'ın aldığı `siginfo_t` bilgisidir.

Örnekte `sa_handler` için `ouch` adlı bir fonksiyon yazılmış. Fonksiyon önce bildirilmiş, sonra kullanılmış ve tanımlanmış. Fonksiyon "Received SIGINT" gibi bir mesaj basar. Kullanım adımları şöyledir:

- `act` adlı yapının içi doldurulur. `act.sa_handler = ouch` yazılır.
- `sigemptyset(&act.sa_mask)` ile maske boş küme yapılır.
- `act.sa_flags = 0` ile bayraklar sıfırlanır.
- `sigaction(SIGINT, &act, NULL)` çağrılır. Yani `SIGINT` için kullanılacak yapının yerine `act` yazılır.

Bundan sonra kesme sinyali geldiğinde bizim doldurduğumuz yapı geçerli olur: boş kümeden oluşan maske, sıfır bayraklar ve `ouch` fonksiyonu.

```
#include <signal.h>
#include <stdio.h>
```

```c
#include <unistd.h>

void ouch(int sig) {
    printf("Received SIGINT (%d)\n", sig);
}

int main(void) {
    struct sigaction act;
    act.sa_handler = ouch;                    /* SIG_IGN yazılırsa SIGINT yok sayılır */
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;
    sigaction(SIGINT, &act, NULL);
    while (1) {
        printf("Hello World!\n");
        sleep(1);
    }
}
```

Program sürekli “Hello World” basarken öğretim elemanı Ctrl+C’ye bastı. Program durmadı, yalnızca işleyicinin mesajı basıldı. Normalde varsayılan eylemle kesme sinyali programı durdururdu, ama artık Ctrl+C ile durdurulamıyordu.

Derlenip çalıştırılan sürümde de aynısı görüldü. Sürece iki kez `SIGINT` gönderildi. Her seferinde “Received SIGINT (2)” basıldı ve “Hello World!” satırları devam etti.

> **Ek bilgi (derste anlatılmadı):** İşleyici içinde `printf` kullanmak gösterim için yaygındır, ama `printf` POSIX’e göre sinyal işleyicide güvenle çağrılabilen (async-signal-safe) fonksiyonlardan değildir. Gerçek programlarda işleyicide `write` kullanılır ya da yalnızca `volatile sig_atomic_t` türünde bir bayrak değiştirilir.

8.4 SIG_IGN ile Yok Sayma

Sonra öğretim elemanı `sa_handler`’a kendi fonksiyonu yerine sistemde tanımlı işleyicilerden `SIG_IGN`’i yazdı. Bu durumda `SIGINT` yok sayılır. Program çalışırken Ctrl+C’ye basıldığında hiçbir şey olmadı. Ctrl+Z’ye ise bir işleyici atanmadığı için o çalıştı ve program Ctrl+Z ile durduruldu.

Denememizde `SIG_IGN` sürümüne `SIGINT` gönderildi. Süreç hiçbir mesaj basmadan çalışmaya devam etti. İşleyicili sürüme `SIGTSTP` (Ctrl+Z) gönderildiğinde ise süreç durduruldu ve `ps`’te `T` durumunda göründü.

8.5 Önceki Eylemi Geri Yükleme

Öğretim elemanı önceki eylemi (previous action) geri yükleme konusunu ayrıntılı anlatmayacağını, ama kendimizin bakabileceğini söyledi. `sigaction`’ın üçüncü parametresine bir `oldact` yapısı verilirse eski eylem orada saklanır. Sonra bu yapı ile yeniden `sigaction` çağrılarak eski eylem geri yüklenir. Bu da sinyal maskesindeki saklayıp geri yükleme mantığının aynısıdır.

Bir de “ilginç” bir örnek gösterdi. Bu örnekte `main`’de `SIGINT` için birinci işleyici kuruluyor. Birinci işleyici çalışınca bir mesaj basıyor ve sinyal için ikinci işleyiciyi kuruyor. İkinci işleyici de üçüncüyü kuruyor. En sonda sinyal varsayılan eyleme çevriliyor, böylece bir sonraki kesme sinyali programı sonlandırabiliyor.

> **Ek bilgi (derste anlatılmadı):** `signal(sig, handler)` daha eski ve basit bir arayüzdür; önceki işleyiciyi dönüş değeri olarak verir. Davranışı sistemden sisteme değiştiği için (örneğin işleyicinin her teslimden sonra varsayılana dönüp dönmediği) POSIX taşınabilir kodda `sigaction` kullanılmasını önerir.

Örnek Sorular

1. (derste sözlü, 00:05:05) `exec` ile `fork`’un farkı nedir?

Cevap (derste verildi): `fork` çağıranın birebir kopyası olan bir çocuk süreç oluşturur. `exec` ise çocuğun farklı bir iş yapmasını, orada bambaşka bir programın çalışmasını sağlar.

> **Ders sonrası not:** `exec` yeni süreç oluşturmaz, çağıran sürecin görüntüsünü değiştirir; PID aynı kalır.

2. (derste sözlü, 00:08:52) `atexit` ile kayıtlı bir fonksiyon, `exit(0)` ile çıkan çocuk süreçte de çağrılır mı?

Cevap (derste verildi): Derste gösterilen kodda kayıt ebeveynde yapılmıştı. Çıktıda temizleyici fonksiyon ebeveyn biterken çağrıldı, çocuk ise `exit(0)` ile normal şekilde bitti.

> **Çözüm (derste verilmedi):** Sonuç kaydın nerede yapıldığına bağlıdır. Derleyerek doğrulandı: `fork`'tan sonra yalnız ebeveyn dalında kaydedilirse fonksiyon bir kez, ebeveynde çalışır. `fork`'tan önce kaydedilirse çocuk da kaydı devralır ve fonksiyon iki kez, iki farklı PID ile çalışır.

3. (derste sözlü, 00:10:22) `return` ile `exit`'in farkı nedir?

Cevap (derste verildi): Öğretim elemanına göre `exit` bir sistem çağrısıdır, `return` bir komuttur.

> **Ders sonrası not:** `exit()` bir kütüphane fonksiyonudur. `atexit` fonksiyonlarını çalıştırır, tamponları boşaltır ve `_exit` sistem çağrısını yapar. `return` yalnız bulunduğu fonksiyondan döner. `main`'den `return` etmek `exit`'e denktir.

4. (derste sözlü, 00:15:50 ve 00:16:36) "Zombi 2" örneğinde çocuk sonsuz `while` döngüsündeyken program neden sonlanmış göründü ve neden zombi oluşmadı? Çocuk döngüden nasıl çıktı?

Cevap (derste kesin verilmedi): Öğretim elemanı nedeni bilmediğini söyledi. Bir öğrenci, çocuk döngüye gelmeden ebeveynin önce çalışıp `exit` yapmış olabileceğini önerdi. Öğretim elemanı ebeveynin çocuk bitmeden çıkabildiğini, öksüz durumun da zaten bu olduğunu söyledi.

> **Çözüm (derste verilmedi):** Derleyerek ve `ps` ile doğrulandı. Çocuk döngüden çıkmamıştır. Sonlanan yalnızca ebeveyndir; kabuk ebeveyni beklediği için komut istemi geri gelir ve program bitmiş gibi görünür. Çocuk öksüz kalır, `R` durumunda çalışmaya devam eder ve yeni ebeveyni `init/systemd` olur. Çocuk hâlâ çalıştığı için zombi değildir. Bittiğinde de yeni ebeveyni onu toplayacağı için zombi kalmaz. Hangi sürecin önce çalıştığı sonucu değiştirmez.

5. (derste sözlü, 00:23:43) Boru (pipe) ile FIFO'nun farkı nedir?

Cevap (derste verildi): Boruya isimsiz boru, FIFO'ya isimli boru denir. İsimsiz boru ebeveyn ile çocuk arasında kullanılır. FIFO'da böyle bir gereklilik yoktur, ilişkisiz süreçler de dosya sistemindeki adı üzerinden kullanabilir.

6. (derste sözlü, 00:27:01) Boru örneğinde çocuk yazıyorsa ebeveyn ne yapmalıdır?

Cevap (derste verildi): Ebeveyn tersini yapar. Yazma ucunu kapatır, okuma ucundan bir `int` boyundaki veriyi okur, okuma ucunu kapatır ve sonucu basar.

7. (derste sözlü, 00:44:02) FIFO örneğinde istemci önce çalıştırılınca iletişim neden kurulmadı?

Cevap (derste verildi): Bunu bir öğrenci fark etti. FIFO'ları sunucu `mkfifo` ile oluşturur, istemcide `mkfifo` yoktur. Sunucu çalışmadan istemci başlatılırsa açılacak FIFO henüz oluşmamıştır. Önce sunucu, sonra istemci çalıştırılmalıdır.

> **Çözüm (derste verilmedi):** Derleyerek doğrulandı. FIFO dosyası yokken istemcideki `open("/tmp/fifo.1", O_WRONLY)` "No such file or directory" hatası verir. Sunucu önce çalıştırıldığında iletişim kuruldu ve dosya içeriği istemcinin ekranına basıldı.

Dersin Toparlaması

Bu uygulama oturumunda süreç yönetiminin C ve Linux tarafı ele alındı. Önce `exec` ile çocuğun bambaşka bir program çalıştırabildiği görüldü. Sonra süreç sonlandırmada `exit(0)`, `atexit` ile çıkışta çalışan fonksiyonlar ve `exit` ile `return` farkı işlendi. Ebeveyn çocuğunun bitiş durumunu almazsa zombi süreç, ebeveyn çocuktan önce biterse öksüz süreç oluştuğu örneklerle gösterildi. Bunların önüne geçmenin yolu olarak `wait` ve `WIFEXITED/WEXITSTATUS` makroları anlatıldı.

İkinci yarıda süreçler arası iletişime geçildi. İsimsiz boru ile ebeveyn-çocuk arasında tek yönlü veri aktarımı, isimli boru (FIFO) ile ilişkisiz iki süreç arasında istemci-sunucu iletişimi kuruldu. Eşzamansız iletişim örneği olarak sinyaller anlatıldı: üç tepki türü, `sigprocmask` ile kritik bölgede sinyal engelleme ve `sigaction` ile kendi işleyicimizi ya da `SIG_IGN`'i kurma. Bir sonraki hafta semaforlar anlatılacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.